1 Мета роботи

Дослідити асиметричні механізми, реалізувати Ed25519 та X25519 у Python. Проаналізувати вибір алгоритмів, типові помилки, постквантову криптографію й ElGamal.

2 Принципи асиметричної криптографії

1. Відкритий ключ передають іншим, закритий зберігають таємно. У шифруванні відкритим ключем захищають дані, закритим відновлюють; у підписі закритим підписують, відкритим перевіряють.

2. Шифрування дає конфіденційність; підпис - перевірку цілісності й походження, але не приховує даних; узгодження - спільний секрет, але без автентифікації не встановлює особу.

3. RSA зі спеціальними схемами шифрує або підписує. ECDSA/EdDSA лише підписують, ECDH/X25519 узгоджують секрет. Різні задачі й гарантії не дозволяють взаємозаміни.

4. RSA та ECC мають різні параметри й складність атак, тому порівнюють рівень стійкості. Орієнтовно RSA-3072 та ECC із 256-бітовими параметрами дають 128-бітовий класичний рівень.

5. Безпечний padding RSA: OAEP для шифрування, PSS для підпису. Відсутній або невідповідний padding допускає атаки попри велику довжину ключа.

6. Секретний одноразовий nonce ECDSA не повторюють для різних повідомлень: це може розкрити ключ. Його формує CSPRNG або стандартний детермінований алгоритм.

7. Асиметричні операції дорогі й обмежують розмір входу. У гібридній схемі дані шифрує симетричний алгоритм, а ключ передає або узгоджує асиметричний.

8. Закритий ключ захищають у сховищі, обмежують доступ і строк дії, відкликають при компрометації. Його не журналюють; резервні копії теж захищають.

3 Практичне дослідження цифрового підпису

Середовище: Python 3.12.0, cryptography 50.0.2, macOS ARM64. Згенеровано ключову пару Ed25519 і підписано повідомлення в UTF-8. Інша незалежна пара використана для негативної перевірки. Основний фрагмент:

```
private = Ed25519PrivateKey.generate()
public = private.public_key()
other = Ed25519PrivateKey.generate().public_key()
signature = private.sign(message)
public.verify(signature, message)
# Наступні виклики дають InvalidSignature:
public.verify(signature, message + b"!")
other.verify(signature, message)
```

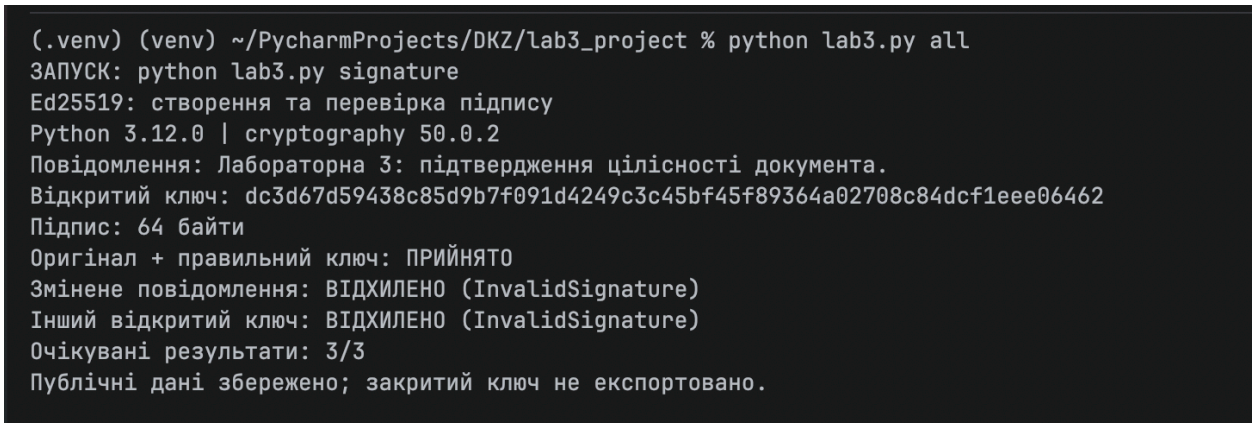


Рисунок 1 - Скриншот виконання досліду Ed25519

Таблиця 1 - Результати перевірки підпису

Умова	Очікування	Отримано
Оригінал і правильний ключ	Прийняття	Прийнято
Повідомлення змінено	Відхилення	InvalidSignature
Інший відкритий ключ	Відхилення	InvalidSignature

Усі три результати відповідають очікуваним. Підпис має 64 байти. Зміна даних порушує перевірку цілісності, а інший ключ не підтверджує цей підпис. Висновок про автора потребує надійного зв’язку відкритого ключа з особою; юридична неспростовність не випливає лише з успішного виклику verify.

4 Практичне узгодження спільного секрету

Alice і Bob незалежно створили пари X25519. Обмін змодельовано серіалізацією та читанням відкритих ключів у пам’яті. Основний фрагмент:

```
alice = X25519PrivateKey.generate()
bob = X25519PrivateKey.generate()
alice_secret = alice.exchange(bob.public_key())
bob_secret = bob.exchange(alice.public_key())
same = hmac.compare_digest(alice_secret, bob_secret)
```

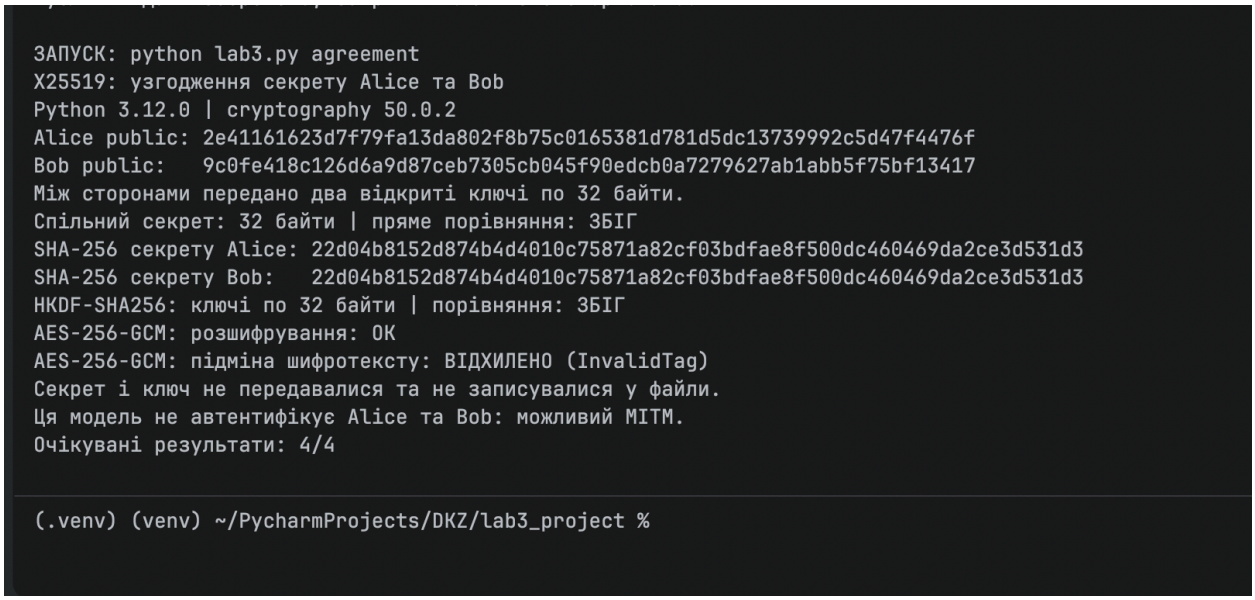


Рисунок 2 - Скриншот виконання досліду X25519

Секрети збігаються завдяки ECDH: застосування обох закритих параметрів до спільної бази дає однаковий результат незалежно від порядку. Порівняно самі 32 байти й додатково їх SHA-256-відбитки.

Обміняно лише відкриті ключі, без спільного секрету. За належних параметрів пасивний класичний спостерігач не може практично відновити секрет. Активна підміна ключів можлива: X25519 не підтверджує особу.

Окремі HKDF-SHA256 з однаковими сіллю та контекстом дали однакові 256-бітові ключі. AES-GCM відновив дані й відхилив підміну. Для захищеної передачі також потрібні nonce і тег; особи учасників ця модель не автентифікує.

5 Вибір механізмів для заданих ситуацій

Таблиця 2 - Механізм алгоритм і обґрунтування

Ситуація	Механізм і приклад	Обґрунтування
1. Авторство та цілісність документа	Цифровий підпис Ed25519	Підписують закритим ключем; перевіряють автентичним відкритим
2. Спільний ключ через недовірену мережу	Автентифікований X25519 + HKDF у TLS 1.3	Секрет узгоджують без передачі. Сертифікат і підпис або попередня довіра до ключа захищають від підміни.
3. Конфіденційний файл 2 ГБ	Гібрид: AES-256-GCM + RSA-OAEP-SHA256 для ключа	Випадковий ключ AES шифрує файл, RSA - лише цей ключ. Відкритий ключ отримувача перевіряють до використання.
4. Зберігання паролів	Парольне гешування Argon2id з унікальною сіллю	Асиметрична криптографія не потрібна. Зберігають сіль, параметри й результат; витрати пам’яті та часу ускладнюють перебір.
5. Оновлення для багатьох пристроїв	Ed25519 для підпису маніфесту оновлення	Виробник захищає закритий ключ, пристрої містять довірений відкритий. Маніфест зв’язує версію й геш образу.

RSA безпосередньо не шифрує 2 ГБ: він має малий допустимий вхід і високу вартість операції. Файл обробляють потоково або автентифікованими частинами, з унікальним nonce для кожної частини під тим самим ключем; перевіряють тег, порядок і повноту файла.

Для оновлень перевірка підпису доповнюється політикою версій проти повернення до старої вразливої версії. Надійність результату залежить також від захисту довіреного ключа на пристрої.

6 Аналіз типових помилок

1. RSA без належного padding. Помилка - пряме використання математичного перетворення або схеми не за призначенням. Наслідки - передбачуваність, підробка чи атаки на розшифрування. Усунення: OAEP для шифрування, PSS для підпису та коректна обробка помилок у перевіреній бібліотеці. Стійкість факторизації не усуває недоліків схеми.

2. Повтор nonce в ECDSA. Два підписи різних повідомлень можуть розкрити одноразовий параметр і закритий ключ. Потрібні якісний CSPRNG або детермінована генерація за RFC 6979, підтримувана бібліотекою; уражений ключ замінюють. Сильна крива не захищає від витоку через повтор параметра.

3. Передбачуваний генератор ключів. Малий простір можливих станів дозволяє перебирати ключі чи відновлювати їх. Використовують генератор криптографічної бібліотеки з системною ентропією, виправляють причину й замінюють слабкі ключі. Велика записана довжина не компенсує брак ентропії.

4. Неперевірена належність відкритого ключа. Нападник підставляє власний ключ, видає себе за учасника й організовує MITM. Перевіряють сертифікат, ім’я та ланцюжок довіри або звіряють відбиток надійним каналом. Правильне шифрування ключем нападника лише захищає дані для нього.

5. Одна пара для різних призначень. Змішування підпису, розшифрування та інших операцій збільшує наслідки витоку й ризик взаємодії протоколів. Створюють окремі пари та політики доступу для кожної функції. Математична коректність алгоритмів не доводить сумісності їх спільного використання.

Отже, надійність системи залежить від схеми, випадковості, автентичності ключів і правил використання. Окремий криптографічний примітив не забезпечує ці умови автоматично.

7 Сучасна постквантова криптографія

1. Квантова загроза. Алгоритм Шора на достатньо потужному квантовому комп’ютері розв’язує факторизацію й дискретний логарифм, на яких ґрунтуються RSA, ECDSA та ECDH. Тому RSA, Ed25519 і X25519 не вважають постквантовими. Ризик для довготривалих таємниць існує й за сценарію «зберегти зараз - розшифрувати пізніше».

2. PQC - криптографічні алгоритми для звичайних комп’ютерів, розраховані на опір відомим класичним і квантовим атакам. Це не квантовий канал зв’язку й не абсолютна гарантія від усіх майбутніх атак.

3. ML-KEM за FIPS 203 встановлює спільний ключ через інкапсуляцію та декапсуляцію. ML-DSA за FIPS 204 створює й перевіряє підписи. Вони виконують різні задачі: перший замінює квантово вразливі засоби встановлення ключів, другий - підпису.

4. У KEM відправник використовує відкритий ключ отримувача й отримує секрет разом із шифротекстом інкапсуляції; отримувач відновлює секрет закритим ключем. Підпис натомість пов’язує повідомлення з власником ключа, не створюючи спільного секрету. KEM сам по собі не автентифікує відправника.

5. Гібридне встановлення поєднує, наприклад, X25519 і ML-KEM. Два незалежно отримані секрети об’єднує визначений протоколом combiner/KDF із потрібним контекстом. Мета - зберегти захист, якщо хоча б один компонент лишається стійким; це потребує правильної конструкції та автентифікації.

6. Перехід відбувається поетапно через сумісність протоколів, обсяг ключів і повідомлень, швидкодію, сертифікати та оновлення пристроїв. Спочатку інвентаризують використання алгоритмів і пріоритетні дані, перевіряють сумісність, потім розгортають підтримувані схеми. Поетапність не означає відкладання підготовки.

8 Індивідуальне завдання ElGamal

1. Призначення. ElGamal забезпечує асиметричне шифрування відкритим ключем адресата та відновлення повідомлення його закритим ключем.

2. Ідея. У циклічній групі відкритий ключ отримують із генератора й секретного показника. Відправник щоразу обирає новий випадковий показник і створює дві компоненти шифротексту. Адресат секретом відтворює маску та відновлює повідомлення.

3. Примітиви. Потрібні операції в циклічній групі, піднесення до степеня, обернення та CSPRNG. Важливі належний вибір групи й складність задач Діффі-Геллмана; складність дискретного логарифма сама не доводить усіх гарантій.

4. Властивості. Потрібні правильність розшифрування й конфіденційність. Випадковість дозволяє отримувати різні шифротексти того самого повідомлення. Базовий ElGamal не гарантує цілісність та автентичність і допускає керовану зміну шифротексту.

5. Приклад. У системі збору конфіденційних заявок дані шифрують відкритим ключем організатора. Практичний захист потребує перевіреної гібридної схеми з автентифікованим шифруванням і перевіркою ключа.

6. Обмеження. Дві групові компоненти збільшують шифротекст. Потрібні коректне кодування й перевірка параметрів. Повтор випадкового показника розкриває зв’язки між повідомленнями. Квантова стійкість відсутня.

9 Висновки

Усі сім перевірок успішні: Ed25519 прийняв оригінал і відхилив дві некоректні умови; X25519 та HKDF дали однакові секрет і ключ; AES-GCM відновив дані й відхилив підміну. Обґрунтовано вибір механізмів, ризики помилок і роль PQC. Для мережі потрібна автентифікація учасників.